Name of Student: VIKAS FOOLCHAND KUSHWAHA			
Roll Number: 29/B		**Lab Practical Number:** 04	
Title of LAB Practical: Implementation of Node.js Timer Functions.			
DOP: 08/10/2026		**DOS:** 12/10/2026	
CO Mapped: CO1	**PO Mapped:** PO3, PO5, PSO1, PSO2	**Faculty Signature:**	**Marks:**

Aim: Implementation of Node.js timer function.

Description:

Node.js provides built-in timer functions to manage asynchronous operations that are executed after a specified time interval. These timers are part of the timers module, which is included by default in Node.js. Timer functions are widely used in various scenarios, including scheduling tasks, implementing delays, and running recurring operations.

Key Timer Functions in Node.js:

1. **`setTimeout(callback, delay, …args)`** - Executes a function callback after a time interval has passed. The time interval (**delay**) is specified by the user in milliseconds.
2. **`setInterval(callback, delay, …args)`** - Execute a function callback at every passing time interval as specified the user.
3. **`setImmediate(callback, …args)`** - Executes a function callback immediately after I/O events in current even loop phase end.
4. **`process.nextTick(callback, …args)`** - Schedules the function to be executed in the next event event loop, before any I/O operations.
5. **`clearTimeout(timeoutId)`** - Stops a timeout that was previously scheduled using setTimeout.
6. **`clearInterval(intervalId)`** - Stops an interval that was previously scheduled by setInterval.

How setInterval Works.

The setInterval function is used to execute the specified callback after every time interval that is specified by the user. It continues to execute it until the user explicitly cancels it by calling the clearInterval function.

```
setInterval(callback, delay, …args);
```

- **callback:** The javascript function that is to be executed by setInterval
- **delay:** Amount of time delay specified in milliseconds that the function waits before the next execution cycle.
- **args:** Optional arguments that are to be passed to the callback function.

Event Loop and Timers.

Timers in Node.js are managed by the Event Loop:

- Timers are placed in the timer phase of the event loop.
- They are executed after their delay has elapsed, but the actual execution depends on the availability of the event loop (e.g., other tasks may delay it).

Practical Applications

- **Real-time Data Logging:** Record server statistics or user activity at regular intervals.
- **Polling Services:** Query APIs at defined time interval for data updates.
- **Task Scheduling:** Automate recurring tasks like sending reports or cleaning up files.

Code:

QUESTION:
Create an application to demonstrate Node.js Functions-timer function(displays every 10 seconds).

CODE

```
function displayMessage() {
    const time = new Date().toLocaleTimeString();
    console.log(`Timer function executed at ${time}`);
}

console.log("Timer started. Press Ctrl+C to stop.");

const timeId = setInterval(displayMessage, 10_000);
```

Output:

```
> node timer.js
Timer started. Press Ctrl+C to stop.
Timer function executed at 3:55:22 PM
Timer function executed at 3:55:32 PM
Timer function executed at 3:55:42 PM
^C
```

Conclusion:

In this Node.js practical, we demonstrated the use of the `setInterval` timer function to execute a task repeatedly at regular time intervals. The program outputs a message and the current time every 10 seconds, showcasing how timers can be used effectively for periodic operations in a Node.js environment. This simple yet powerful feature is widely applicable in scenarios like logging, polling, or scheduling tasks in server-side applications.